CERTIFICATE

*This is to certify that Mr. **Rohit Malviya** of **M.Sc. D.A. Part-II Semester IV** has completed the practical work in the Subject of **Artificial Intelligence** during the academic year 2025-26 under the guidance of Prof. **Sanjana Bhangale** being the partial requirement for the fulfillment of the curriculum of **Degree of Master of Science in Data Analytics, University of Mumbai**.*

Place:

Date:

Name & Signature of faculty *Name & Signature of external*

INDEX

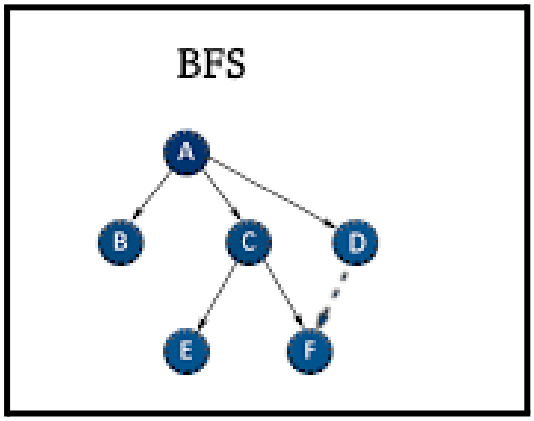
Sr. no.	Details	Date	Signature
1.	Write a program to implement Breadth First Search using Python.		
2.	Write a program to implement Depth First Search using Python.		
3.	Write a program to implement Problem Solving Agent to find the best route using Python.		
4.	Write a program to implement an 8-puzzle problem using Python.		
5.	Write a program to implement Tic-Tac-Toe using Python.		
6.	Write a program to implement a Water Jug problem using Python.		
7.	Write a program to implement Traveling Salesman using Python.		
8.	Write a program to implement Tower of Hanoi using Python.		
9.	Write a program to implement Missionaries and Cannibals problems using Python.		

MSc Data Analytics Part II/4816

Practical 1

Aim: Write a program to implement Breadth First Search using Python.

Code:




```
graph={
    'A':['B','C','D'],
    'B':['A'],
    'C':['A','E','F'],
    'D':['A','F'],
    'E':['C'],
    'F':['C','D']
}
def bfs(node):
  visited=[False]*(len(graph))
  queue=[]
  visited.append(node)
  queue.append(node)
  while queue:
    v=queue.pop(0)
    print(v,end=" ")
    for neigh in graph[v]:
      if neigh not in visited:
        visited.append(neigh)
        queue.append(neigh)

if __name__=="__main__":
  bfs('A')
```

Output:

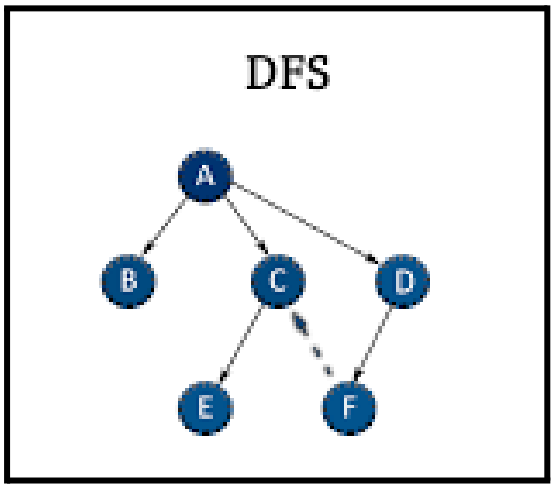
```
A B C D E F
```

MSc Data Analytics Part II/4816

Practical 2

Aim: Write a program to implement Depth First Search using Python.

Code:




```
graph={
    'A':['B','C','D'],
    'B':['A'],
    'C':['A','E','F'],
    'D':['A','F'],
    'E':['C'],
    'F':['C','D']
}

visited=set()

def dfs(visited,graph,node):
    if node not in visited:
        print(node,end="")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited,graph,neighbour)
dfs(visited,graph,'A')
```

Output:

```
ABCEFD
```

MSc Data Analytics Part II/4816

Practical 3

Aim: Write a program to implement Problem Solving Agent to find the best route using Python.

Code:

```
# Function to perform Best First Search
from heapq import heappush, heappop
def bestFirstSearch(edges, src, target, n):
    # create the adjacency list
    adj = [[] for _ in range(n)]
    for edge in edges:
        adj[edge[0]].append([edge[1], edge[2]])
        adj[edge[1]].append([edge[0], edge[2]])
    # create a visited array to
    # keep track of visited nodes
    visited = [False] * n
    # create the min heap to store the nodes
    # based on the cost
    pq = []
    # push the source node in the min heap
    heappush(pq, [0, src])
    # mark the source node as visited
    visited[src] = True
    # to store the path
    path = []
    # loop until the min heap is empty
    while pq:
        # get the top element of the min heap
        x = heappop(pq)[1]
        # push the current node in the path
        path.append(x)
        # if the current node is the target node
        # break the loop
        if x == target:
            break
        # loop through the edges of the current node
        for edge in adj[x]:
            if not visited[edge[0]]:
                # mark the node as visited
                visited[edge[0]] = True
                # push the node in the min heap
                heappush(pq, [edge[1], edge[0]])
    return path
```

```python
if __name__ == "__main__":
    n = 14
    edgeList = [
        [0, 1, 3], [0, 2, 6], [0, 3, 5],
        [1, 4, 9], [1, 5, 8], [2, 6, 12],
        [2, 7, 14], [3, 8, 7], [8, 9, 5],
        [8, 10, 6], [9, 11, 1], [9, 12, 10],
        [9, 13, 2]
    ]
    source = 0
    target = 9
    path = bestFirstSearch(edgeList, source, target, n)
    for i in range(len(path)):
        print(path[i], end=" ")
```

Output:

```
0 1 3 2 8 9
```

MSc Data Analytics Part II/4816

Practical 4

Aim: Write a program to implement an 8-puzzle problem using Python.

Code:

```
import heapq
GOAL_STATE=(1,2,3,4,5,6,7,8,0)
MOVES=[(-1,0),(1,0),(0,-1),(0,1)]
def manhattan_distance(state):
    distance=0
    for i in range(9):
      if state[i]==0:
        continue
      target_row, target_col = divmod(state[i]-1,3)
      curr_row,curr_col=divmod(i,3)
      distance+=abs(target_row-curr_row)+abs(target_col-curr_col)
    return distance

def a_star(start):
  open_list=[]
  heapq.heappush(open_list,(0+manhattan_distance(start), 0, start,[]))
# Added '0' for initial 'g' value
  visited=set()
  while open_list:
    f,g,state,path=heapq.heappop(open_list)
    if state==GOAL_STATE:
      return path
    if state in visited: # Added this check to avoid redundant
processing
        continue
    visited.add(state)
  zero_pos=state.index(0)
  zero_row,zero_col=divmod(zero_pos,3)
  for move in MOVES:
    new_row=zero_row+move[0]
    new_col=zero_col+move[1]
    if 0<=new_row<3 and 0<=new_col<3:
      new_zero_pos=new_row*3+new_col
      new_state=list(state)
      new_state[zero_pos],new_state[new_zero_pos]=new_state[new_zero_po
s],new_state[zero_pos]
      new_state=tuple(new_state)
      # if new_state not in visited: # This check is now redundant
because of the 'if state in visited' check above
```

```
        new_g=g+1
        new_f=new_g+manhattan_distance(new_state)
        # Pushed 4 values (f, g, state, path) into the heap
       heapq.heappush(open_list,(new_f,new_g,new_state,path+[new_state]))
  return None

def print_puzzle(state):
  for i in range(0,9,3):
    print(state[i:i+3])

if __name__=="__main__":
  start_state=(1,2,3,8,0,4,7,6,5)
  print("Initial State: ")
  print_puzzle(start_state)

  solution=a_star(start_state)
  if solution:
    print("\nSolution steps: ")
    for step in solution:
      print_puzzle(step)
      print("----")
  else:
    print("No solution found.")
```

Output:

```
Initial State:
(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
No solution found.
```

MSc Data Analytics Part II/4816

Practical 5

Aim: Write a program to implement Tic-Tac-Toe using Python.

Code:

```
import numpy as np
import random
from time import sleep

def create_board():
    return(np.array([[0, 0, 0],
                     [0, 0, 0],
                     [0, 0, 0]]))

def possibilities(board):
    l=[]
    for i in range(len(board)):
        for j in range(len(board)):
            if board[i][j]==0:
                l.append((i,j))
    return(l)

def random_place(board,player):
    selection = possibilities(board)
    current_loc=random.choice(selection)
    board[current_loc]=player
    return(board)

def row_win(board,player):
    for x in range(len(board)):
        win=True
        for y in range(len(board)):
            if board[x,y]!=player:
                win=False
                continue
        if win==True:
            return(win)
    return(win)

def col_win(board,player):
    for x in range(len(board)):
        win=True
        for y in range(len(board)):
            if board[y][x]!=player:
```

```
                win=False
                continue
        if win==True:
            return(win)
    return(win)

def diag_win(board,player):
    win=True
    y=0
    for x in range(len(board)):
        if board[x,x]!=player:
            win=False
    if win:
        return win
    win=True
    if win:
        for x in range(len(board)):
            y=len(board)-1-x
            if board[x,y]!=player:
                win=False
    return win

def evaluate(board):
    winner =0
    for player in [1,2]:
        if (row_win(board,player) or
            col_win(board,player) or
            diag_win(board,player)):
            winner = player
    if np.all(board!=0)and winner==0:
        winner=-1
    return winner

def play_game():
    board,winner,counter=create_board(),0,1
    print(board)
    sleep(2)

    while winner==0:
        for player in[1,2]:
            board=random_place(board,player)
            print("Board after "+str(counter)+" move")
            print(board)
```

```
            sleep(2)
            counter+=1
            winner=evaluate(board)
            if winner !=0:
                break
    return winner

print("Winner is: " +str(play_game()))
```

Output:

```
[[0 0 0]
 [0 0 0]
 [0 0 0]]
Board after 1 move
[[0 0 1]
 [0 0 0]
 [0 0 0]]
Board after 2 move
[[0 0 1]
 [0 0 0]
 [0 0 2]]
Board after 3 move
[[0 0 1]
 [0 1 0]
 [0 0 2]]
Board after 4 move
[[0 0 1]
 [0 1 0]
 [0 2 2]]
Board after 5 move
[[1 0 1]
 [0 1 0]
 [0 2 2]]
Board after 6 move
[[1 0 1]
 [0 1 2]
 [0 2 2]]
Board after 7 move
[[1 1 1]
 [0 1 2]
 [0 2 2]]
Winner is: 1
```

Practical 6

Aim: Write a program to implement a Water Jug problem using Python.

Code:

```
from collections import deque
def is_goal_state(jug1,jug2,z):
  """Check id we have reached the goal state where either jug has 'z'
liters of water"""
  return jug1==z or jug2==z
def bfs_water_jug_problem(capacity1,capacity2,target):
  """Solve the water jug problem using BFS"""
  queue=deque([(0,0)])
  visited=set()
  visited.add((0,0))
  actions=[
      ('Fill Jug1',lambda jug1,jug2: (capacity1,jug2)),
      ('Fill Jug2',lambda jug1,jug2: (jug1,capacity2)),
      ('Empty Jug1',lambda jug1,jug2: (0,jug2)),
      ('Empty Jug2',lambda jug1,jug2: (jug1,0)),
      ('Pour from Jug1 to Jug2',lambda jug1,jug2:
(max(0,jug1-(capacity2-jug2)),min(capacity2,jug2+jug1))),
      ('Pour from Jug2 to Jug1',lambda jug1,jug2:
(min(capacity1,jug1+jug2),max(0,jug2-(capacity1-jug1)))),
  ]
  while queue:
    jug1,jug2=queue.popleft()
    if is_goal_state(jug1,jug2,target):
      print(f"Goal state reached: ({jug1},{jug2})")
      return True
    for action,operation in actions:
      new_jug1,new_jug2=operation(jug1,jug2)
      if (new_jug1,new_jug2) not in visited:
        visited.add((new_jug1,new_jug2))
        queue.append((new_jug1,new_jug2))
        print(f"State: ({new_jug1},{new_jug2}) - Action: {action}")
  print("No solution found.")
  return False


capacity1=int(input("Enter the capacity of jug1: "))
capacity2=int(input("Enter the capacity of jug2: "))
target=int(input("Enter the target amount of water: "))
bfs_water_jug_problem(capacity1,capacity2,target)
```

Output:

```
Enter the capacity of jug1: 4
Enter the capacity of jug2: 3
Enter the target amount of water: 2
State: (4,0) - Action: Fill Jug1
State: (0,3) - Action: Fill Jug2
State: (4,3) - Action: Fill Jug2
State: (1,3) - Action: Pour from Jug1 to Jug2
State: (3,0) - Action: Pour from Jug2 to Jug1
State: (1,0) - Action: Empty Jug2
State: (3,3) - Action: Fill Jug2
State: (0,1) - Action: Pour from Jug1 to Jug2
State: (4,2) - Action: Pour from Jug2 to Jug1
State: (4,1) - Action: Fill Jug1
Goal state reached: (4,2)
True
```

Practical 7

Aim: Write a program to implement Traveling Salesman using Python.

Code:

```
import itertools
import math

def calculate_route_distance(route,dist_matrix):
  total_distance=0
  for i in range(len(route)-1):
    total_distance+=dist_matrix[route[i]][route[i+1]]
  total_distance+=dist_matrix[route[-1]][route[0]]
  return total_distance

def tsp_brute_force(dist_matrix):
  num_cities=len(dist_matrix)
  cities=list(range(num_cities))
  all_permutations=itertools.permutations(cities)
  min_distance=float('inf')
  best_permutation=None
  for perm in all_permutations:
    distance=calculate_route_distance(perm,dist_matrix)
    if distance<min_distance:
      min_distance=distance
      best_permutation=perm
  return best_permutation,min_distance

if __name__ =="__main__":
  dist_matrix=[
      [0,10,15,20],
      [10,0,35,25],
      [15,35,0,30],
      [20,25,30,0]
  ]
  best_route,min_distance=tsp_brute_force(dist_matrix)
  print("Best Route:",best_route)
  print("Minimum Distance:",min_distance)
```

Output:

```
Best Route: (0, 1, 3, 2)
Minimum Distance: 80
```

Practical 8

Aim: Write a program to implement Tower of Hanoi using Python.

Code:

```
def TowerOfHanoi(n, fromRod, toRod, auxRod):
    if n == 0:
        return
    TowerOfHanoi(n-1, fromRod, auxRod, toRod)
    print("Disk", n, " moved from ", fromRod, " to ", toRod)
    TowerOfHanoi(n-1, auxRod, toRod, fromRod)

if __name__ == "__main__":
    n = 3

    # A, C, B are the name of rods
    TowerOfHanoi(n, 'A', 'C', 'B')
```

Output:

```
Disk 1  moved from  A  to  C
Disk 2  moved from  A  to  B
Disk 1  moved from  C  to  B
Disk 3  moved from  A  to  C
Disk 1  moved from  B  to  A
Disk 2  moved from  B  to  C
Disk 1  moved from  A  to  C
```

MSc Data Analytics Part II/4816

Practical 9

Aim: Write a program to implement Missionaries and Cannibals problems using Python.

Code:

```
def is_valid_state(m_left,c_left,m_right,c_right):
  if m_left < 0 or c_left < 0 or m_right< 0 or c_right <0:
    return False

  if (m_left > 0 and m_left < c_left) or (m_right > 0 and m_right <
c_right):
    return False
  return True

def dfs(state,goal_state,path,visited):
  m_left,c_left,boat_position=state
  if state==goal_state:
    return path+[state]
  if state in visited:
    return None
  visited.add(state)
  if boat_position == 0:
    possible_moves=[
      (1,0),(2,0),(0,1),(0,2),(1,1)
  ]
  else:
    possible_moves=[
        (-1,0),(-2,0),(0,-1),(0,-2),(-1,-1)
    ]
  for m_move,c_move in possible_moves:
    new_m_left=m_left-m_move
    new_c_left=c_left-c_move
    new_m_right=3-new_m_left
    new_c_right=3-new_c_left
    new_boat_position=1-boat_position
    if is_valid_state(new_m_left,new_c_left,new_m_right,new_c_right):
      new_state=(new_m_left,new_c_left,new_boat_position)
      result=dfs(new_state,goal_state,path+[state],visited)
      if result:
        return result

  return None
```

```
def solve_missionaries_and_cannibals():
  initial_state=(3,3,0)
  goal_state=(0,0,1)
  visited=set()
  solution=dfs(initial_state,goal_state,[],visited)
  if solution:
    print("Solution found:")
    for step in solution:
      print(f"Missionaries Left:{step[0]},Cannibals left:{step[1]},Boat
Position:{'Left' if step[2]==0 else 'Right'}")
  else:
    print("No solution exists.")

solve_missionaries_and_cannibals()
```

Output:

```
Solution found:
Missionaries Left:3,Cannibals left:3,Boat Position:Left
Missionaries Left:3,Cannibals left:1,Boat Position:Right
Missionaries Left:3,Cannibals left:2,Boat Position:Left
Missionaries Left:3,Cannibals left:0,Boat Position:Right
Missionaries Left:3,Cannibals left:1,Boat Position:Left
Missionaries Left:1,Cannibals left:1,Boat Position:Right
Missionaries Left:2,Cannibals left:2,Boat Position:Left
Missionaries Left:0,Cannibals left:2,Boat Position:Right
Missionaries Left:0,Cannibals left:3,Boat Position:Left
Missionaries Left:0,Cannibals left:1,Boat Position:Right
Missionaries Left:1,Cannibals left:1,Boat Position:Left
Missionaries Left:0,Cannibals left:0,Boat Position:Right
```